

Express.js Middleware — Key Points

The Core Architecture of Express

Source: <https://expressjs.com/en/guide/using-middleware.html>

What is Middleware?

Express is fundamentally a routing and middleware web framework. An Express application is essentially a series of middleware function calls. Middleware functions have access to the request object (req), the response object (res), and the next middleware function in the pipeline (next).

The Pipeline & next()

Middleware executes sequentially. If the current middleware function does not end the request-response cycle (e.g., by calling `res.send()` or `res.json()`), it **MUST** call `next()` to pass control to the next middleware function. If it forgets to call `next()`, the request will hang indefinitely.

Application-Level vs Router-Level

- Application-level middleware is bound to the 'app' object using `app.use()` or `app.METHOD()`. It executes for every request matching the path.
- Router-level middleware works identically but is bound to an instance of `express.Router()`, allowing you to modularize routes across files.

Error-Handling Middleware

Express identifies error-handling middleware by its arity (number of arguments). It **MUST** have exactly four arguments: (err, req, res, next). If you pass an argument to `next(err)` anywhere in the pipeline, Express skips all regular middleware and jumps straight to the next error-handling middleware.

Built-in Body Parsers

Unlike FastAPI, Express does not parse request bodies by default. You must register built-in middleware to populate `req.body`:

- `express.json()`: Parses incoming requests with JSON payloads.

- `express.urlencoded()`: Parses URL-encoded payloads (form submissions).